



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Sebastian Alexander Villeda Reyna **Carné:** 1032625

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R// Es un conjunto de reglas y protocolos que permite que diferentes aplicaciones de software se comuniquen entre sí a través de internet de manera sencilla y eficiente
2. ¿Qué es un endpoint?
R// Es una dirección digital específica (una URL) en un servidor donde una aplicación puede conectarse para enviar o recibir datos
3. ¿Qué diferencia existe entre una URL y un endpoint?
R// Una URL es cualquier dirección web general que sirve para ubicar un recurso en internet, a diferencia del endpoint que es una URL específica que esta combinada con un método HTTP, el cual está diseñado exclusivamente para una app o API.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R//
GET: Pide y recupera datos sin modificar nada.
POST: Se encarga de enviar los datos nuevos al servidor para crear registros.
PUT: Actualiza el dato que se desee, lo elimina por completo y lo reemplaza por el que se quiera.
DELETE: Borra el dato/registro indicado.
5. ¿Qué es JSON y por qué es común en las APIs?
R// Es un formato de texto ligero basado en pares de clave-valor y listas ordenadas que sirve para almacenar y transferir datos estructurados.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R// Es una respuesta numérica de tres dígitos que el servidor envía al navegador o cliente para indicar el resultado de una solicitud.

El 200 indica que la solicitud del cliente se ha completado con éxito y el servidor devuelve el recurso solicitado. Como datos en forma JSON.

El 404 significa que el servidor no encontró el recurso que se andaba buscando, un ejemplo es a la hora de colocar una URL y que está este mal escrita.

El 500 señala que hay un error genérico en el servidor, ya que indica que el servidor encontró una situación inesperada y no pudo completar la solicitud.
7. ¿Qué información se puede encontrar en la respuesta de una API?
R// Contiene la información que el servidor devuelve tras procesar una solicitud hecha por el cliente.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R// Tiene como ventajas probar y simular una API antes de escribir el código de programación ofrece grandes ventajas en el desarrollo de software.



9. ¿Qué es y para qué sirve Swagger UI?

R// Es una herramienta de código abierto que genera una interfaz web interactiva y visual a partir de la especificación OpenAPI de una API REST.

Sirve como documentación visual automática, pruebas en tiempo real, facilita la colaboración y ahorro de tiempo.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

R// Funciona como una plataforma integral para diseñar, enviar solicitudes, probar, documentar y gestionar APIs a lo largo de todo su ciclo de vida.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
/pet/{petId}
- Indique el método HTTP utilizado.
GET
Indique qué parámetro necesita la solicitud.
Necesita el parámetro **petId**. Es un parámetro de ruta (*Path Parameter*), lo que significa que el valor se pasa directamente embebido dentro de la URL de la solicitud (por ejemplo, https://petstore.swagger.io/v2/pet/10326).
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
200 OK (*al consultar un ID existente en el sistema*).
- Explique con sus palabras qué información contiene la respuesta.
La respuesta contiene un objeto estructurado en formato **JSON** que detalla toda la información asociada a la mascota consultada. Incluye:
id: El número identificador único de la mascota.
category: Un objeto con el ID y nombre de la categoría a la que pertenece (por ejemplo, *Perros*).
name: El nombre asignado a la mascota (por ejemplo, *Firulais Cósmico*).
photoUrls: Una lista o arreglo con los enlaces (URLs) de las imágenes de la mascota.
tags: Una lista de etiquetas o descriptores asignados a la mascota.
status: El estado actual del recurso en la tienda (available, pending o sold).

Evidencia: captura de Swagger mostrando la operación y su respuesta.



petstore3.swagger.io/#/pet/addPet

Parameters

No parameters

Request body **required** application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 10001,
  "category": {
    "id": 1,
    "name": "Perros"
  },
  "name": "Firulais Cósmico",
  "photoUrls": [
    "https://example.com/photo.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Rescatado"
    }
  ],
  "status": "available"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10001,
    "category": {
      "id": 1,
      "name": "Perros"
    },
    "name": "Firulais Cósmico",
    "photoUrls": [
      "https://example.com/photo.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "Rescatado"
      }
    ],
    "status": "available"
  }'
```

petstore3.swagger.io/#/pet/addPet

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10001,
    "category": {
      "id": 1,
      "name": "Perros"
    },
    "name": "Firulais Cósmico",
    "photoUrls": [
      "https://example.com/photo.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "Rescatado"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 10001,
  "category": {
    "id": 1,
    "name": "Perros"
  },
  "name": "Firulais Cósmico",
  "photoUrls": [
    "https://example.com/photo.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Rescatado"
    }
  ],
  "status": "available"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PATCH
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 179
content-type: application/json
date: Fri, 02 Oct 2026 02:09:00 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	Successful operation	No links



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
R// Se utilizó el método **PUT**. En la arquitectura REST, este método se emplea para actualizar o reemplazar completamente la representación de un recurso existente en el servidor utilizando su identificador único (id).
- ¿Qué datos fueron enviados en el body?
R// Se envió un objeto en formato **JSON** en el cuerpo (*body*) de la solicitud, manteniendo el mismo id registrado previamente (10326), pero modificando los atributos name (de "Firulais Cósmico" a "Firulais Guardián") y status (de "available" a "sold"):

```
{
  "id": 10326,
  "category": {
    "id": 1,
    "name": "Perros"
  },
  "name": "Firulais Guardián",
  "photoUrls": [
    "https://example.com/photo.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Rescatado"
    }
  ],
  "status": "sold"
}
```
- ¿Qué código HTTP devolvió la API?
La API devolvió el código **200 OK**, indicando que la solicitud de actualización fue procesada y aplicada exitosamente en el servidor.
- ¿Cómo comprobó que el cambio realmente ocurrió?
Se comprobó realizando una nueva consulta de lectura mediante la solicitud GET /pet/10326. En la respuesta devuelta por la API se constató que los valores de los campos "name" y "status" habían cambiado efectivamente a "Firulais Guardián" y "sold", respectivamente.



The screenshot shows the Swagger UI interface for the petstore3 API. The endpoint selected is `POST /pet/updatePet`. The request body is a JSON object representing a pet with the following details:

```
{
  "id": 18326,
  "category": {
    "id": 1,
    "name": "Perros"
  },
  "name": "Firula Guardian",
  "photoUrls": [
    "https://example.com/photo.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Rescatado"
    }
  ],
  "status": "sold"
}
```

The response is a 200 status with the same JSON object and the following headers:

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 195
content-type: application/json
date: Fri, 02 Oct 2026 02:10:36 GMT
server: Jetty(9.4.53.v20231009)
```

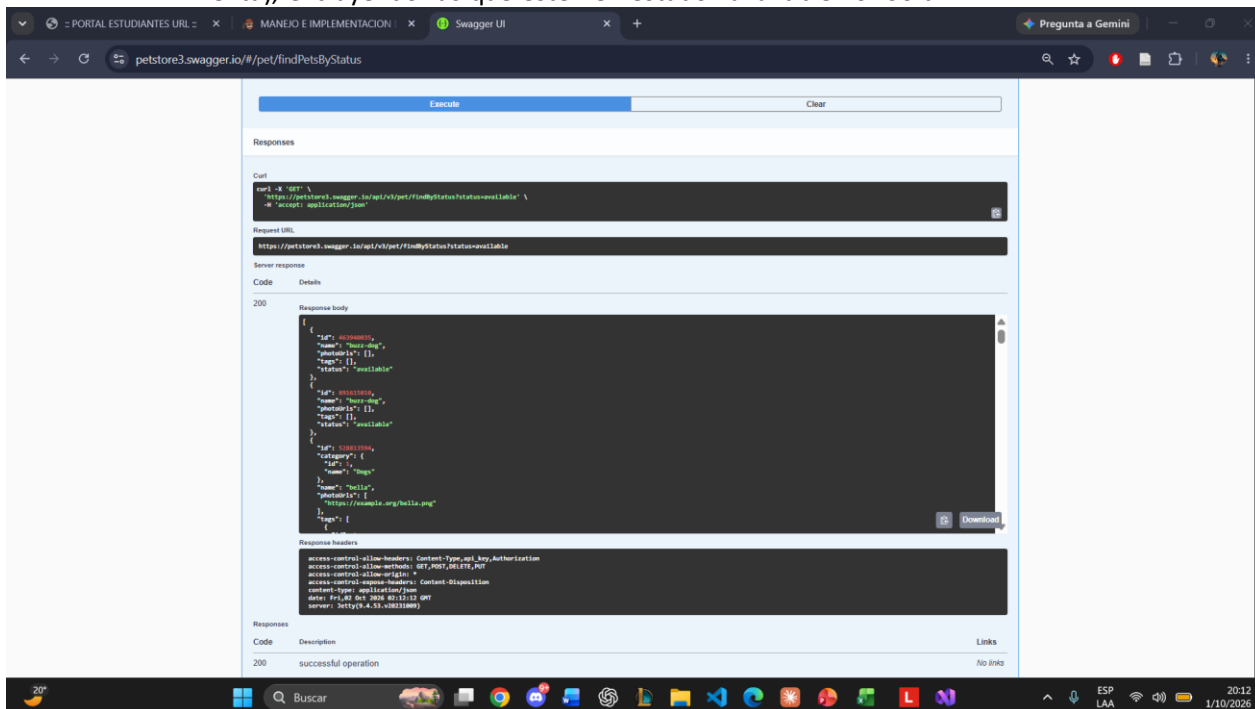
5. Consultar utilizando parámetros

- Utilice `GET /pet/findByStatus` y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
El parámetro utilizado es **status**, al cual se le asignó el valor **available**.
- Explique dónde se envía ese parámetro.
Se envía en la URL como un parámetro de consulta (*Query Parameter*). En la estructura de la solicitud HTTP, se ubica al final de la ruta del endpoint precedido por un signo de interrogación (?), de la forma:



https://petstore.swagger.io/v2/pet/findByStatus?status=available

- Observe la estructura de la respuesta.
La respuesta es una lista o arreglo JSON (Array), delimitado por corchetes [...]. Dentro de esta lista, cada elemento es un objeto JSON { ... } que contiene los atributos completos de una mascota (id, category, name, photoUrls, tags, status) cuyo estado coincide con el criterio filtrado (available). Si no se encuentran registros en la base de datos en ese momento, la API devuelve un arreglo vacío [] acompañado de un código HTTP 200 OK.
- Indique qué sucede si cambia el valor del parámetro por pending.
La API aplica un nuevo filtro sobre la base de datos de la tienda y devuelve únicamente el arreglo de objetos JSON correspondiente a las mascotas cuyo estado es "pending" (pendientes de adopción o venta), excluyendo las que estén en estado "available" o "sold".



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



The screenshot shows the VS Code interface with the REST Client extension. The 'POST' request is defined with a URL 'https://petstore.swagger.io/v2/pet' and a JSON body. The response is also shown in the 'Body' tab.

```
POST http://localhost:3000/api/pets POST New Request
{
  "id": 10326,
  "category": {
    "id": 1,
    "name": "Perros"
  },
  "name": "Firulaís C3smico",
  "photoUrls": [
    "https://example.com/photo.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Rescatado"
    }
  ],
  "status": "available"
}
```

The screenshot displays the Swagger UI for a pet store API. The top navigation bar includes links for 'POST http://loc', 'GET Link 1', 'GET Link 2', 'Mia_Semana', 'POST New Req', and 'GET New Req'. The main header shows the method 'GET' and the URL 'https://petstore.swagger.io/v2/pet/10326'. Below the header, the 'Query Params' section is empty. The 'Body' tab is selected, showing a JSON response with a 200 OK status. The JSON body contains the following data:

```

1 {
2   "id": 10326,
3   "category": {
4     "id": 1,
5     "name": "Pezros"
6   },
7   "name": "Firulaís C3smico",
8   "photoUrls": [
9     "https://example.com/photo.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Rescatado"
15    }
16  ],
17  "status": "available"
18 }

```

The bottom status bar indicates 'Ready to go' and shows system information like '20°C', 'ESP LAA', and the time '20:23 1/10/2026'.



PUT:

The screenshot shows a Postman interface with a PUT request to `https://petstore.swagger.io/v2/pet`. The request body is a JSON object representing a pet. The response is a 200 OK status with a JSON body containing the pet's details and a new tag.

```
1 {
2   "id": 10326,
3   "category": {
4     "id": 1,
5     "name": "Perros"
6   },
7   "name": "Firulais Guardián",
8   "photoUrls": [
9     "https://example.com/photo.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Rescatado"
15    }
16  ],
17  "status": "sold"
18 }
```

Body: `{ "id": 10326, "category": { "id": 1, "name": "Perros" }, "name": "Firulais Guardián", "photoUrls": [ "https://example.com/photo.jpg" ], "tags": [ { "id": 1, "name": "Rescatado" } ], "status": "sold" }`

GET:

The screenshot shows a Postman interface with a GET request to `https://petstore.swagger.io/v2/pet/findByStatus?status=available`. The response is a 200 OK status with a JSON array of two pets.

```
1 [
2   {
3     "id": 37,
4     "category": {
5       "id": 0,
6       "name": "Sonny"
7     },
8     "name": "Updated name",
9     "photoUrls": [
10      "string"
11    ],
12    "tags": [
13      {
14        "id": 0,
15        "name": "string"
16      }
17    ],
18    "status": "available"
19  },
20  {
21    "id": 470015619,
```



Delete:

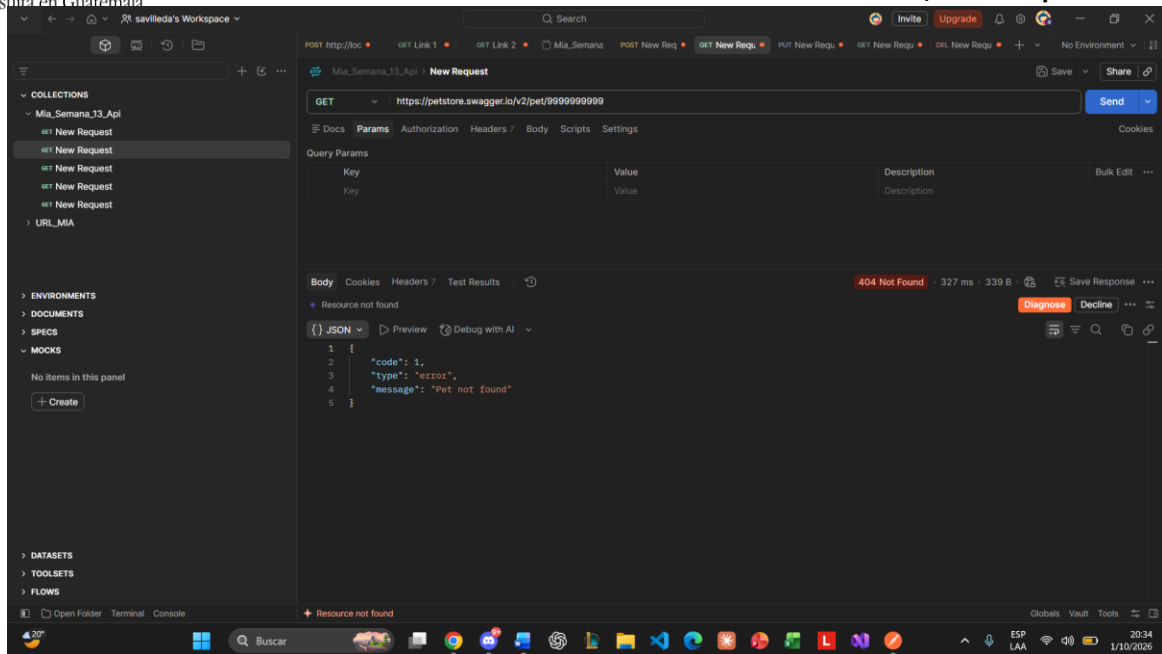
The screenshot shows the Postman interface with a DELETE request to `https://petstore.swagger.io/v2/pet/10326`. The response is `200 OK` with a status of `300 ms` and `335 B`. The response body is JSON: `{\"code\": 200, \"type\": \"unknown\", \"message\": \"10326\"}`.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

- Consultar un ID inexistente.
Solicitud realizada. R// `https://petstore.swagger.io/v2/pet/9999999999`
- Código HTTP recibido. R// `404 Not Found`
- Respuesta de la API. R// `{\"code\": 1, \"type\": \"error\", \"message\": \"Pet not found\"}`
- Qué esperaba que ocurriera. R// Que el servidor notificara que el recurso buscado no existe en el sistema.
- Qué ocurrió realmente. R// Devuelve un código de estado `404 Not Found` indicando la ausencia del registro.
- Explicación de la causa R// El identificador proporcionado en la ruta URL no coincide con ningún registro activo en la base de datos de la API.



2. Enviar un JSON incorrecto o incompleto.

Solicitud realizada.

R// <https://petstore.swagger.io/v2/pet>

g) Código HTTP recibido.

R// 400 Bad Request (o 500 Internal Server Error dependiendo de la versión del parser).

h) Respuesta de la API.

R// Mensaje indicando fallo al deserializar/procesar el cuerpo del mensaje (*bad input / parse error*)

i) Qué esperaba que ocurriera.

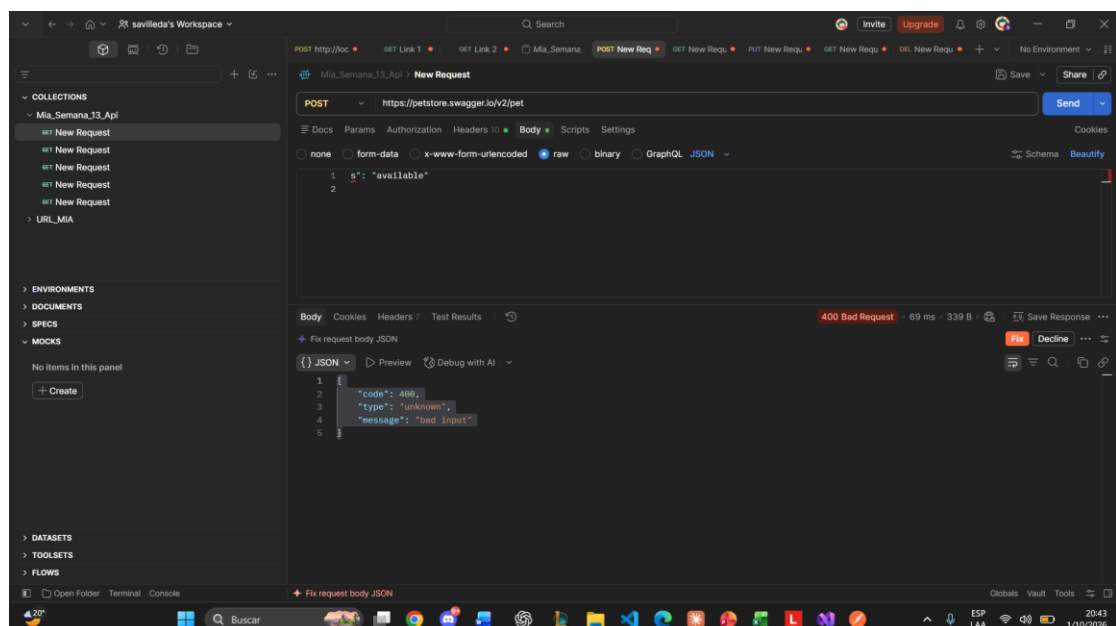
R// Que la API rechazara la solicitud al no cumplir con el estándar JSON.

j) Qué ocurrió realmente.

R// El servidor bloqueó el procesamiento de la petición al no poder interpretar la estructura del cuerpo.

k) Explicación de la causa

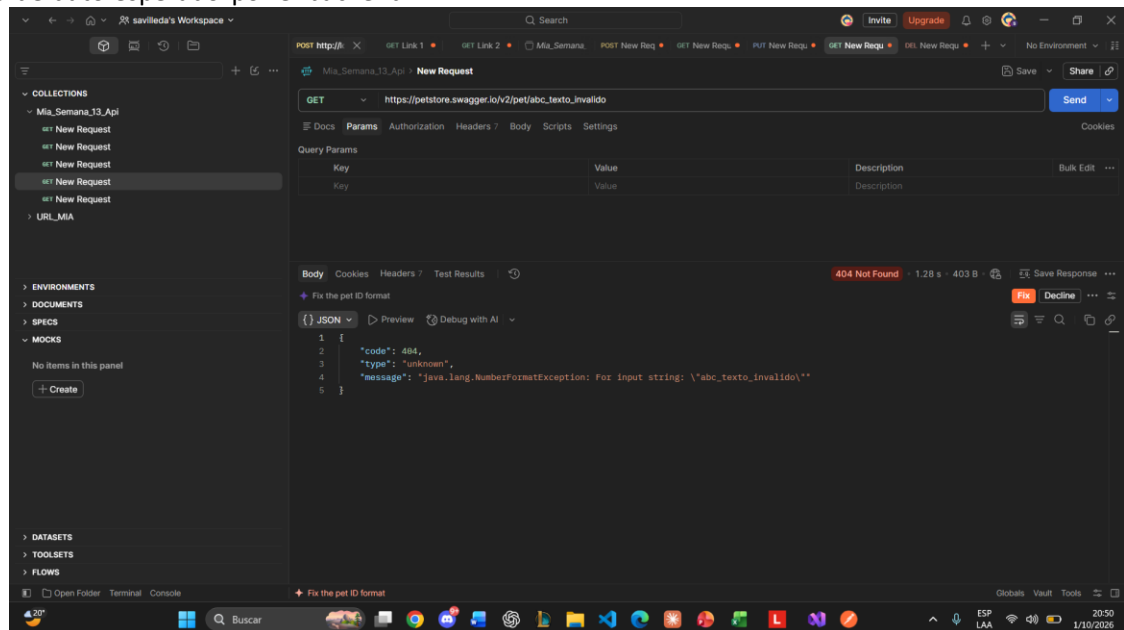
R// Falta de sintaxis JSON válida (comas, comillas o llaves faltantes), lo que impide que el servidor lea los datos del recurso.





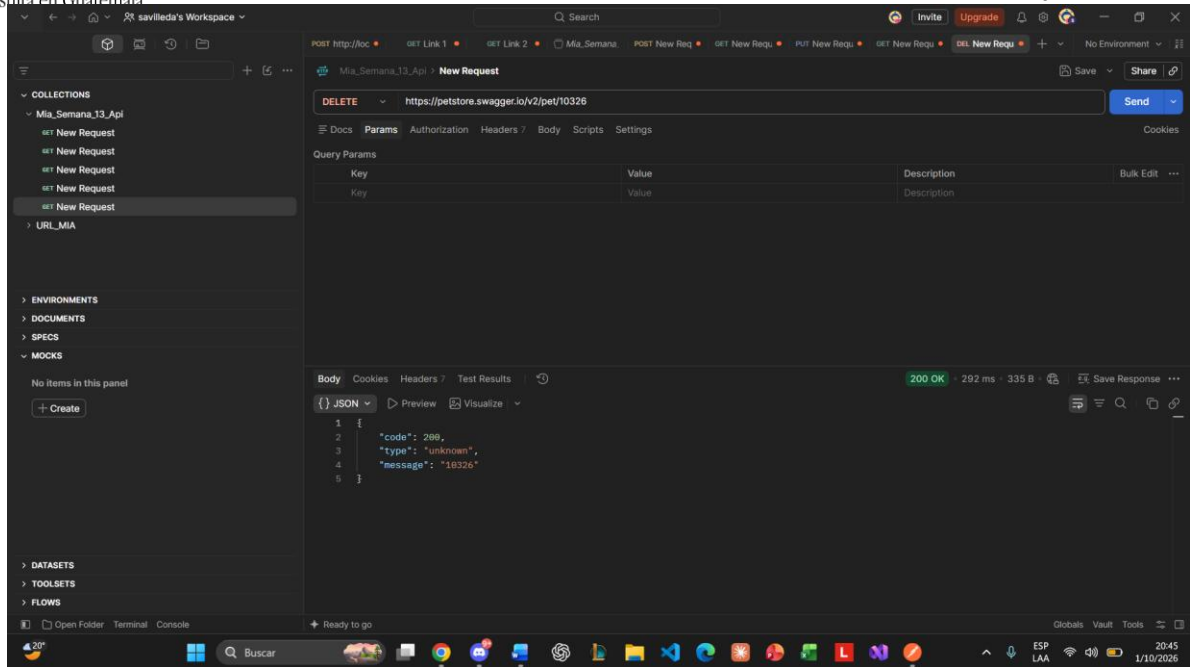
Utilizar un parámetro con un valor no esperado.

- l) Solicitud realizada.
R// `https://petstore.swagger.io/v2/pet/abc_texto_invalido`
- m) Código HTTP recibido.
R// 404 Not Found (o 400 Bad Request según la validación del servidor).
- n) Respuesta de la API.
R// `{"code": 404, "type": "unknown", "message": "java.lang.NumberFormatException: For input string: \"abc_texto_invalido\"}"`
Qué esperaba que ocurriera.
R// El servidor procesó la petición, intentó convertir la cadena de texto a un número entero para buscar el ID, devolvió una excepción de formato de número (*NumberFormatException*) y arrojó un código de respuesta de error.
Explicación de la causa
R// El parámetro de ruta `petId` está definido en la especificación OpenAPI de Petstore como un tipo numérico de 64 bits (integer). Al enviar letras o símbolos, la API no puede castear/convertir el valor al tipo de dato esperado por el backend.



Intentar eliminar un recurso que ya no existe

- o) Solicitud realizada.
R// `https://petstore.swagger.io/v2/pet/10326`
- p) Código HTTP recibido.
R// 404 Not Found
- q) Respuesta de la API.
R// Mensaje de recurso no encontrado o respuesta vacía.
- r) Qué esperaba que ocurriera.
R// Que la API devuelva un error indicando que el elemento a borrar ya no existe.
- s) Qué ocurrió realmente.
R// Retorna 404 Not Found.
- t) Explicación de la causa
R// La primera ejecución borró el objeto de la base de datos, por lo que las solicitudes posteriores para eliminar ese mismo ID fallan



Para cada prueba registre:

- u) Solicitud realizada.
- v) Código HTTP recibido.
- w) Respuesta de la API.
- x) Qué esperaba que ocurriera.
- y) Qué ocurrió realmente.
- z) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.



POST http://loc GET Link 1 GET Link 2 Mia_Semana POST New Req GET New Requ PUT New Requ GET New Requ DEL New Requ + No Environment

Mia_Semana_13_Api > New Request

POST https://petstore.swagger.io/v2/pet

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "id": 10326,
3   "category": {
4     "id": 1,
5     "name": "Perros"
6   },
7   "name": "Mascota Detective",
8 }
9
```

Body Cookies Headers 7 Test Results 200 OK 425 ms 469 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 10326,
3   "category": {
4     "id": 1,
5     "name": "Perros"
6   },
7   "name": "Mascota Detective",
8   "photoUrls": [
9     "https://example.com/photo.jpg"
10  ],
11   "tags": [
12     {
13       "id": 1,
14       "name": "Laboratorio"
15     }
16  ],
17   "status": "available"
18 }
```

Open Folder Terminal Console Ready to go

20° Buscar

20:54 1/10/2026

POST http://loc GET Link 1 GET Link 2 Mia_Semana POST New Req GET New Requ PUT New Requ GET New Requ DEL New Requ + No Environment

Mia_Semana_13_Api > New Request

GET https://petstore.swagger.io/v2/pet/10326

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results 200 OK 511 ms 469 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 10326,
3   "category": {
4     "id": 1,
5     "name": "Perros"
6   },
7   "name": "Mascota Detective",
8   "photoUrls": [
9     "https://example.com/photo.jpg"
10  ],
11   "tags": [
12     {
13       "id": 1,
14       "name": "Laboratorio"
15     }
16  ],
17   "status": "available"
18 }
```

Open Folder Terminal Console Ready to go

20° Buscar

20:55 1/10/2026



The screenshot displays the Postman API client interface. The top section shows a PUT request to `https://petstore.swagger.io/v2/pet` with a raw JSON body. The request is successful, returning a 200 OK status. The response body is a JSON object representing a pet with the following details:

```
{  "id": 10326,  "category": {    "id": 1,    "name": "Perros"  },  "name": "Mascota Detective",  "photoUrls": [    "https://example.com/photo.jpg"  ],  "tags": [    {      "id": 1,      "name": "Laboratorio"    }  ],  "status": "sold"}
```

The bottom section shows a DELETE request to `https://petstore.swagger.io/v2/pet/10326`. The request is also successful, returning a 200 OK status. The response body is a JSON object with the following details:

```
{  "code": 200,  "type": "unknown",  "message": "10326"}
```



The screenshot shows the Postman application interface. On the left, the 'COLLECTIONS' panel is expanded, showing a collection named 'Mia_Semana_13_Api' with several 'GET New Request' items. The main workspace displays a 'New Request' for a GET method to the URL 'https://petstore.swagger.io/v2/pet/10326'. The 'Body' tab is selected, showing a JSON response:

```
{  "code": 1,  "type": "error",  "message": "Pet not found"}
```

. The status bar at the bottom indicates '404 Not Found' with a response time of 81 ms and a size of 339 B. The Windows taskbar at the bottom shows the date as 1/10/2026 and the time as 20:59.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.